Evaluation of quadrat-related functions

Gabriel Arellano

2026-07-14

Contents

1 Goal

There is a set of functions in the CTFS / fgeo functionality that seems related to handling quadrats and coordinates in ForestGEO plots. These are the types of location information in this type of datasets:

- gx and gy: global coordinates within the plot, relative to a (0, 0) corner
- lx and ly: local coordinates within the quadrat
- col and row: column and row within the plot
  - conceptually, just coarse x and y coordinates
  - in practice, could be named with letters, start at 00 or 01, etc.
- quadrat name, following certain conventions, typically derived from row and col
- hectares (rarely)
- quadrat index, numerical
- quadrats of different resolution than the default
  - typically smaller quadrats nested within larger quadrats
  - sometimes with specific naming that refers to location within larger quadrat (e.g. 5x5 m quadrats, useful for fieldwork and filling maps, within 20x20 quadrats used for data analyses and summaries)

This is the evaluation of the functionality included in the old CTFS distribution of scripts, and the newer fgeo package version:

- In the old CTFS distribution there are 20 functions. Most of them translate from one form (and resolution) of location into another. Besides, there are functions related to identifying neighbors, selecting quadrats, and other related tasks. Probably secondary in importance. I evaluate the functions in the CTFS `quadfunc.r` script and, possibly, their downstream dependencies.

- In the newer `fgeo` package, the functionality has been incorporated through functions that add new columns to data tables, in the `add_var.R` and `add_subquad.R` scripts. It seems this represents an improvement, at least in terms of consistency.

In this document I will often refer to the old script-based distribution of code as "CTFS", and the newer formal R packages as `ctfs` (a more formal distribution of CTFS code) and `fgeo` (same goals, different structure, heavy reorganization).

2 Load code

Older CTFS code is within local scripts. We declare paths to the project (watched by git), load a specific version of the code, and auxiliary functions.

```
# This is my local path to the project (being watched by git)
# You may have to adapt this.
path <- normalizePath(system2("git", c("rev-parse", "--show-toplevel"), stdout = TRUE)[1])
knitr::opts_knit$set(root.dir = path) # global wd, instead of within-chunk wd as setwd()

# Load packages:
library(igraph, quietly = TRUE)
```

```
Warning: package 'igraph' was built under R version 4.4.3
```

```

Attaching package: 'igraph'
```

```
The following objects are masked from 'package:stats':

    decompose, spectrum
```

```
The following object is masked from 'package:base':

    union
```

```
# pak::pak("forestgeo/ctfs")
# pak::pak("forestgeo/fgeo")
library(ctfs, quietly = TRUE)
```

```

Attaching package: 'ctfs'
```

```
The following objects are masked from 'package:igraph':

    neighbors, print_all
```

```
library(fgeo, quietly = TRUE)
```

```
-- Attaching packages ---------------------------------------- fgeo 1.1.4 --

v fgeo.analyze 1.1.15     v fgeo.tool    1.2.10
v fgeo.plot    1.1.11     v fgeo.x       1.1.4

-- Conflicts ------------------------------------------------ fgeo_conflicts() --
x fgeo.analyze::abundance() masks ctfs::abundance()
x fgeo.tool::filter()       masks stats::filter()
```

```
# Source useful functionality:
source(paste0(path, "/evaluation/0-code-to-source.r"))

# Then load CTFS / fgeo2 scripts:

# For reproducibility, this file does code evaluation at a specific
# point in time. Otherwise it won't make sense as the project
# moves forward and the code gest updated.

# The following lines create a temporary file and gives its path.
# You have to clean that at the end of the file, otherwise
# you will accumulate temporary folders somewhere.

DATE = "2026-05-29"
commit = commit_from_date(path = path, date = DATE)
historical_path <- make_dir_to_point_in_history(path = path, commit = commit)
```

This will source the necessary scripts; note that I am loading a specific version of them, by using the path to a specific commit (**historical_path**, linked to commit = 8b1c13366dba84a757fb08ebd9cf5d813c0d1668).

```
modules <- c("01_utilities", "02_biomass", "03_spatial", "04_abundance_diversity",
             "05_demography_growth", "06_topography_habitat", "07_mapping_plotting")

scripts <- unique(unlist(lapply(modules, function(m) {
```

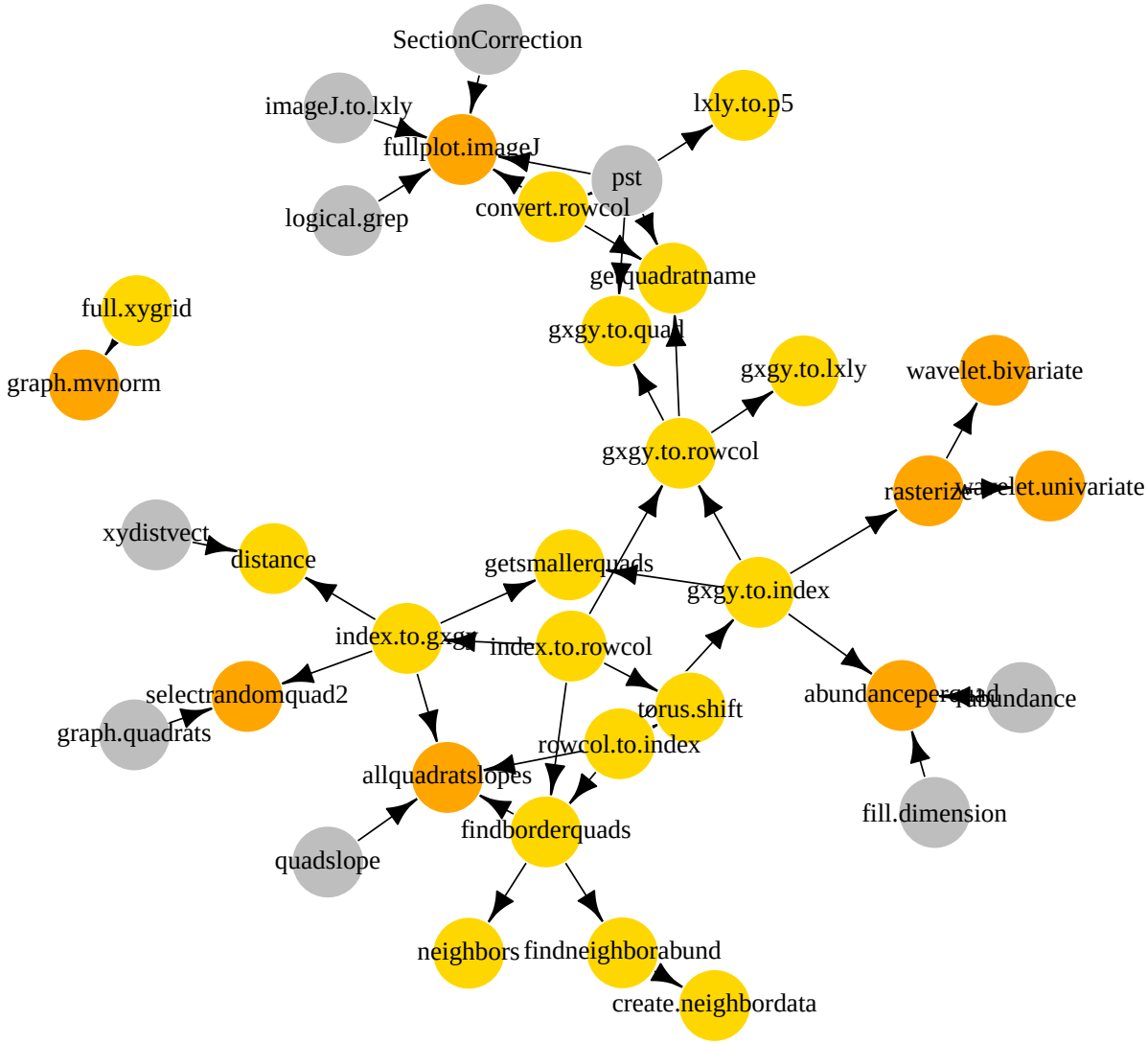
```
  list.files(paste0(historical_path, "/", m, "/R/new"),
             full.names = TRUE, pattern = "\\.R$", ignore.case = TRUE)
})))

# Source scripts, then new:
for(f in scripts) source(f)
```

3 Map dependencies

This maps the mutual dependencies among the CTFS functions:

```
# All of the dependencies among all functions in the global environment:
edges <- get_all_dependencies()
edges <- edges[, c("from_function", "to_function")]
colnames(edges) <- c("from", "to")

# Indentify the target subset of functions and their dependents:
where <- grep("quadfunc.r", scripts)
quadrat_functions <- functions_in_script(scripts[where])
down <- unique(unlist(lapply(quadrat_functions, function(q) downstream_dependents(q, edges))))
down <- down[!down %in% quadrat_functions]
target <- c(quadrat_functions, down)

# Graph representation:
keep <- edges$from %in% target | edges$to %in% target
edges <- edges[keep,,drop = FALSE]

g <- graph_from_data_frame(edges, directed = TRUE)
g.ctfs <- g # copy, just in case
```

```
# Visualize
V(g)$color <- rep("gray", length(V(g)))
V(g)$color[V(g)$name %in% quadrat_functions] <- "gold"
V(g)$color[V(g)$name %in% down] <- "orange"

plot(g, layout = layout.fruchterman.reingold,
     vertex.label = V(g)$name,
     vertex.color= V(g)$color,
     vertex.frame.color= "white",
     vertex.label.color = "black",
     edge.color="black")
```



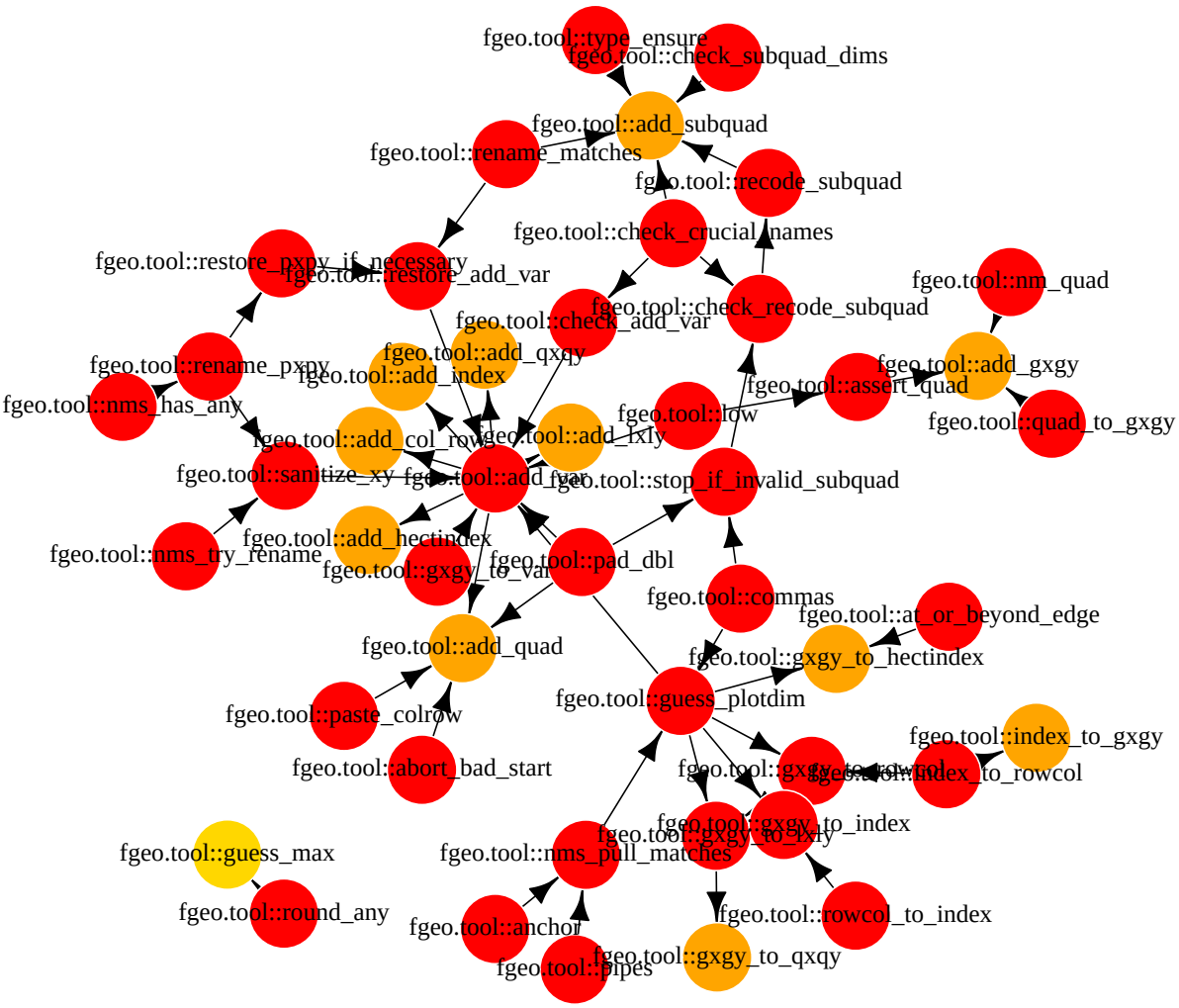
The following code maps the dependencies in the ctfs + fgeo distribution. Because I know the functionality may be hidden somewhere else, I also map the upstream dependencies:

```
# This gets dependencies from a set of packages:
edges <- get_all_dependencies(
  packages = c("fgeo", "fgeo.tool", "fgeo.x", "fgeo.analyze", "fgeo.plot", "ctfs"),
  include_global = FALSE,
  internal = TRUE)

edges <- edges[, c("from_id", "to_id")]
```

```r
colnames(edges) <- c("from", "to")

# Our target functions are distributed in different scripts:
s1 = "https://raw.githubusercontent.com/forestgeo/fgeo.tool/HEAD/R/add_var.R"
s2 = "https://raw.githubusercontent.com/forestgeo/fgeo.tool/HEAD/R/add_subquad.R"
s3 = "https://raw.githubusercontent.com/forestgeo/fgeo.tool/HEAD/R/from_var_to_var.R"
s4 = "https://raw.githubusercontent.com/forestgeo/fgeo.tool/HEAD/R/guess_plotdim.R"

quadrat_functions <- c(functions_in_script(s1),
                       functions_in_script(s2),
                       functions_in_script(s3),
                       functions_in_script(s4))

quadrat_functions <- paste0("fgeo.tool::", quadrat_functions)

# Maybe others? It is difficult to understand fgeo...
patterns <- c("lxly", "qxqy", "gxgy", "pxpy",
              "rowcol", "row_col", "colrow", "col_row",
              "quad",
              "hect")

add1 <- edges$from[unlist(lapply(patterns, function(p)
  grepl(p, edges$from) & !grepl("ctfs::", edges$from)))]
add2 <- edges$to[unlist(lapply(patterns, function(p)
  grepl(p, edges$to)   & !grepl("ctfs::", edges$to)))]

quadrat_functions <- unique(c(quadrat_functions, add1, add2))

# Upstream and downstream links:
down <- unlist(lapply(quadrat_functions, function(q) downstream_dependents(q, edges)))
up <- unlist(lapply(quadrat_functions, function(q) upstream_dependencies(q, edges)))

# Graph representation:
# (This is more restrictive than above, because the high density
# of dependencies in fgeo would it make if difficult to visualize).
keep1 <- edges$from %in% quadrat_functions & edges$to %in% down
keep2 <- edges$from %in% up                & edges$to %in% quadrat_functions
keep <- keep1 | keep2
edges <- edges[keep,,drop = FALSE]

g <- graph_from_data_frame(edges, directed = TRUE)
g.fgeo <- g # copy, just in case
```

```
# Add colors:
V(g)$color <- rep("gray", length(V(g)))
V(g)$color[V(g)$name %in% quadrat_functions] <- "gold"
V(g)$color[V(g)$name %in% down] <- "orange"
V(g)$color[V(g)$name %in% up]   <- "red"
```

```
# Visualize
plot(g, layout = layout.fruchterman.reingold,
     vertex.label = V(g)$name,
     vertex.color= V(g)$color,
     vertex.frame.color= "white",
     vertex.label.color = "black",
     edge.color="black")
```



4 Evaluation of CTFS

The evaluation of CTFS helps to identify the tasks that these functions are supposed to do, and the potential problems with the ways the task is being solved.

In CTFS old code, this seems a relatively well-defined set of functions with surprisingly few dependents and only one direct dependency (`pst()` function, which is a CTFS version of paste/collapse).

4.1 Goal and structure of most functions

As said above, most functions seem to "translate" from one form and resolution of location information to another. The names of the functions inform about the possible ways of expressing location:

- gx and gy: global coordinates
- lx and ly: local coordinates
- quadrat name
- quadrat index
- row and column (of quadrats)

There are somewhat cryptic mentions to smaller quadrats in the names of `getsmallerquads()` and `lxly.to.p5()` functions. No functions depend on them, so it is possible that they will be removed or substituted by more general ways of splitting into quadrats of arbitrary resolution.

It seems most of the difficult things require quadrat indexes, which is not something users will be familiar with, as the data tables typically store quadrat names, the fieldwork is done using quadrat names, etc. This, in itself, makes anything based on quadrat indexes difficult to de-bug or scrutinize, and thus it is a point of vulnerability.

There is an important naming convention for quadrats: CCRR, where CC are two digits for the column (location along x), and RR are two digits for the row (location along y). Both CC and RR could start at 00 or 01, depending on the site-level convention.

- the CCRR convention applies to BCI and other places, but it is not a universally accepted convention, so this is clearly a tricky point.
- the CCRR convention can be different for quadrats of different size, e.g. 5x5 m quadrats within 20x20 m quadrats only use one digit for (internal) column and one digit for (internal) row.
- the specification varies between CTFS functions, sometimes start = 0, sometimes start = "zero".
- the specific CCRR convention should be trivial to guess from the complete data table, but other conventions for naming quadrats may not be that obvious.

The CCRR using numerical digits is a typical point of vulnerability in ForestGEO datasets, as it looks like a number, but it is not a number. Excel often goes from "0133" to "133". This creates issues again and again. Condit code has warnings related to that, things like "Must rewrite if three digit quadrats". The function `convert.rowcol()` adds initial zeros.

Interestingly, the `index.to.rowcol()` function returns numerical rows and columns, that always start with 1. So we can find datasets with row = 1, col = 1, and quadrat = 0000 in CCRR convention with start = 0.

There are two systems to uniquely identify quadrats within this code:

- quadrat name or code, following CCRR form
- quadrat index: numerical, one per quadrat. This is used internally to many of these functions, but does not directly affects important functionality (but check wavelet analysis, and imageJ-related code).

In both cases, the location of the quadrats is implicit! Not explicit! The only way to make quadrat location explicit is to use `quad.to.gxgy()` or `index.to.gxgy()` but that is misleading, as it derives from the assumptions themselves.

The functions apply to trees or quadrats, with quadrats being represented by their corners. For example:

- `quad.to.gxgy()`: returns global coordinates of the bottomleft coordinate of each quadrat, given a CCRR column-row naming convention, and a given gridsize (= quadrat size).
- `gxgy.to.quad()`: returns the quadrat membership of the trees, given their global coordinates.

There is a weird convention in Condit's code when coordinates are unknown, where quadrat = 9999 instead of quadrat = NA. In contrast, `gxgy.to.index()` returns quadrat index = NA when there is not enough information to determine location.

4.2 Neighboring-related functions

There is a small set of functions related to identifying and comparing with neighboring quadrats… Their purpose is not clear, and it does not seems something people would use broadly. Possibly related to null models?:

- `create.neighbordata()`: risky and confusing. Takes as input an "abundanceperquad" object, which follows certain conventions (species in rows, data sorted by quadrat index). Clear candidate for removal. Nothing depends on this. The input is probably the output of the `abundanceperquad()` function, but the dependency is implicit, not explicit. It uses `findneighborabund()` (next).
- `findneighborabund()`: takes similar input, but called "abundvect" now. It finds neighboring quadrats using `findborderquads()` (next), and then copy-pastes abundances from the abundance vector.
- `findborderquads()`: calculates indexes of neighboring quadrats, for a given quadrat index. Probably useful, although using this just to calculate species abundances seems like a narrow application.

- `neighbors()`: very bad name, super general word, for an extremely narrow application. This function takes a vector of absence/presence for all quadrats, and returns proportion of presences among the neighboring quadrats. It is difficult to understand why this has become part of the broadly distributed CTFS package.
- `torus.shift()`: Creates a torus-shifted quadrat topographic dataset. This can be safely removed if a standard torus translation (or constrained, for that matter) is integrated into the pack of functions. Any quadrat-based map can be translated just by translating the corners, then going back to (null) quadrat codes, if needed.

Although not directly connected, there is an interesting function `distance()` that calculates the distances between quadrats. It only wraps from quadrat indexes to coordinates, and then applies `xydistvect()` function in utilities > geometry. Using distances seems a more general approach to identify neighboring quadrats, generalizable to k = 4, k = 8, and so on, using standard k-NN routines.

4.3 Other functions

`getsmallerquads()`: this is a dead-end function, nothing depends on it. It returns small quadrats within larger quadrats, given two resolutions. The smaller resolution will be adjusted internally. It seems a rather arbitrary sub-division of quadrats, instead of directly dividing the entire plot (or any point dataset, for that matter) into quadrat membership at any resolution.

4.4 Depending functions

There are different pats of CTFS code that depend on the quadrat-related functions. So, if anything changes, the depending code must be adjusted accordingly (assuming it will be kept).

First, they relate with `fullplot.imageJ()`. That function is part of a practical routine to go from paper maps to digital coordinates. Still waiting to understand whether this is needed now that people work with tablets and all that.

Second, `gxgy.to.index()` and `index.to.gxgy()` relate to, apparently, rather trivial functions to pick quadrats or summarize abundance by quadrat: `abundanceperquad()` and `selectrandomquad2()`. Clearly, these tasks don't need dedicated functions. The user must be able to summarize abundances by any combination of things, e.g. [quadrat x species] or [family x size class x habitat], or whatever. Something similar can be said for `selectrandomquad2()`.

- Note: `selectrandomquad2()` does **not** pick a random quadrat among the available quadrats. Instead, it returns the coordinates of a square within the plot of a given size. It uses `index.to.gxgy()` internally as a trick, assuming 1x1 m grid. It seems

something useful to do random visualizations of how the forest looks like at local scales, using `graph.quadrats()` internally. Will be reviewed later, but clearly weak candidate to remain as it is.

Third, `gxgy.to.index()` is called by functions that run the wavelet analysis applied to ForestGEO datasets. This is probably something work isolating and developing by itself. Ideally, it should be generalize and integrated with other spatial null models. In any case, it does not seem a desirable or necessary dependency.

Fourth, there are links to a topograhical function: `allquadratslopes()`.

Finally, there is an isolated pair of functions that should be examined together:

- `full.xygrid()` (in the quadfunc.r script) is equal to `base::expand.grid()`.
- `graph.mvnorm()` (in utilities > distributions) apparently is just used to graph contours on multivariate normal distributions). Maybe something to move to the mapping + plotting module.

5 Evaluation of fgeo

The fgeo package splits the functionality in very small pieces. The structure of dependencies is not possible to navigate for the average user. Apparently simple things like `add_quad()` have 20 upstream dependencies only within `fgeo.tool` package. One of the goals of this project is to simplify these dependencies.

Unlike other parts of `fgeo`, the code is not wrapping `ctfs` = CTFS. In fact, the `ctfs` counterpars are not being called at all. Therefore, I have to identify the working code to see if there are differences or something extra in `fgeo` that is not in CTFS original distribution.

5.1 Main functions

Standard cases

The interface with the user takes the form of `add_*()` functions in `fgeo.tool`. Most of them call to `add_var()` which in turns calls to `gxgy_to_var()` which in turns calls a range of functions of the form `gxgy_to_*()`, using global coordinates as the most flexible stepping stone:

- `add_col_row()` calls `add_var()` who calls `gxgy_to_var()` who calls `gxgy_to_rowcol()` who calls two things:
  - `gxgy_to_index()`: calculates row+col and then calls `rowcol_to_index()`, which is working code equal to `rowcol.to.index()` in CTFS; and then

  - `index_to_rowcol()`: working code equal to `index.to.rowcol()` in CTFS. Clearly unnecessary, as rows and columns had to be calculated before.

- `add_quad()` calls `add_col_row()` and then does a trivial wrapping of `paste()` through `fgeo.tool::paste_colrow()`. The working code is same as above.
- `add_hectindex()` calls `gxgy_to_hectindex()`, which is working code = `gxgy.to.hectindex()` in CTFS.
- `add_index()` calls `add_var()` who calls `gxgy_to_var()` who calls `gxgy_to_index()`, which calculates row+col and then calls `rowcol_to_index()`, which is working code equal to `rowcol.to.index()` in CTFS.
- `add_lxly()` calls `add_var()` who calls `gxgy_to_var()` who calls `gxgy_to_lxly()`, which is working code that uses `gxgy_to_rowcol()` inside, which in turns calls other things (see above). The working `fgeo.tool::gxgy_to_lxly()` function does the same as `gxgy.to.lxly()` in CTFS.
- `add_qxqy()` calls `add_var()` who calls `gxgy_to_var()` who calls `gxgy_to_qxqy()` with just changes names of the output of `gxgy_to_lxly()`, which is the working code (see above).
- `add_gxgy()` calls to `quad_to_gxgy()`, which is equal to `quad.to.gxgy()` in CTFS.

In short: there is nothing new in these functions except the sanity checks. The working code is copy-pasted from CTFS scripts without formal dependency.

There are important things to learn, however:

- Some functions exist only to change column names: this is a huge limitation of the approach based on adding columns. Better code should output something that the user could plug into the data using any column name they want.
- The structure of the code based on long chains of calls seems extremely difficult to improve or adapt to other cases or networks out of ForestGEO conventions.

Special case: sub-quadrats

The sub-quadrat case is treated as special, both in fgeo and CTFS. The function `fgeo.tool::add_subquad()` is a version of CTFS function `lxly.to.p5()`. Unlike the other functions above, it is not simply copy-pasted. Instead, it is a complete re-writing with important generalizations:

- it does not assume 20 x 20 m quadrats
- it does not assume 5 x 5 m subquadrats
- it allows the start = 0 or start = 1 version, similar to the quadrat naming convention
- does all the necessary prior checks

The cost in readability to achieve these improvements, however, is huge. It is very cryptic and incorporates an odd vs not-odd distinction internally that did not exist with the original code. It also has extra dependencies, making its behavior almost impossible to anticipate.

In any case, treating this subquadrat-into-quadrat case as a special case seems a weak point of both CTFS and fgeo. **Splitting quadrats into subquadrats is not really different than splitting a plot into subplots = quadrats**. Both things are the same thing, they just require (x, y) coordinates at whatever level it is, and a IDing convention for the resulting pieces. I will keep this in mind when designing new code.

5.2 Auxiliary functions

There are around 20 auxiliary functions, with inter-dependencies. I won't review these functions in detail now. I will come back later to get ideas for warnings and checks, depending on the shape the code takes.

A recurrent auxiliary function is `guess_plotdim()`, which provides a default in case the plotdim parameter (= xmax and ymax) is not provided. The function won't be used as it is (as usual in the package, it calls four other functions, creating dependencies, including conflicting namespace with `scales` and `ggplot2` packages). But the guessing itself may be necessary as a default: `ceiling(max(data$gx, na.rm = TRUE))` and `ceiling(max(data$gy, na.rm = TRUE))`. In the new code, it is probably better to include the guesses for xmin and ymin too, so this can be applied to splits of subplots or portions of the plot.

5.3 Not included in fgeo

The fgeo package does not include functionality to find neighboring quadrats or doing summaries on them, neither functionality to split the entire plot into quadrats of arbitary size (that remains an open issue in the github repository to date).

6 Design principles for new code

Based on the evaluation and overall goals, these are some reflections and design principles for the new code.

6.1 Best practices

If the project only had tree-level data, the only piece of information really needed is just the global coordinates of trees, {gx, gy}. If the project has quadrat-level data, there is a need for quadrat-level IDs and at least one quadrat-level table. The most important information in such quadrat-level table is quadrat location. The link with other tables is made as with any other table: through shared keys = quadrat-level ID. Without any extra assumptions.

Storing location implicit in a quadrat ID is a bad practice. If quadrat location is implicit in quadrat ID, one cannot specify {quadrat IDs, locations} without changing their codes or indexes. This is a great point of vulnerability. A major improvement would be to move from [rigid quadrat IDs with location implicit] to [arbitrary quadrat IDs + explicit locations].

In general, in a well-managed project, quadrat-level IDs could take any form. The new code should be able to work under *any* way of identifying quadrats uniquely, regardless of whether that is a numerical code, or alpha-numerical, or whatever. It must not rely on any quadrat ID convention in particular.

The step of finding location from ID must be rare, separated, explicit. It is just a hack, and it is a risky operation in general. Because of that, it should be OK to make it hard for the user: asking for many details, making it interactive, etc. It is not something that should be done again and again in analytical pipelines or by default within other functions.

To locate quadrats, (xmin, ymin, xmax, ymax) should be preferred over (x, y), as there could be ambiguity regarding left-bottom corner or centroid or something else. This is closer to polygon language, and easier to make compatible with other ways of dividing plots into pieces. It would work for square or rectangular, does not need additional "grid size" information, it won't matter if the quadrats overlap with each other, etc.

For a given team or network, a general recommendation is: get quadrat-level {xmin, ymin, xmax, ymax} coordinates, store them, use best practices from then on, and you won't need any of the functions we are discussing here at all, neither you will have to remember why you named the quadrats in a certain way, etc. You would relate with quadrat-level info the same way you relate with species-level info, habitat-level info, etc: through explicit unique IDs and explicit information stored in tables, not conventions. This general recommendation should be clear and strong in the documentation / tutorials.

6.2 Fieldwork and other situations

The good practices for data storage and linking tables with each other are clear. And, or course, all the problems are rather trivial if one already has a data table with global coordinates.

However, because fieldwork often records progress by quadrat, there are many instances where {quadrat, local coordinates} go before {global coordinates}. In general: **we cannot avoid the problem of extracting {global coordinates} from {quadrat naming convention,**

quadrat ID, local coordinates}. That must be faced and solved. This is also the opportunity for a "fresh start" for those who still store quadrat location implicit in quadrat IDs and not explicitly as they should.

There are cases where the team has obtained or stored quadrat-level information using different quadrat-level IDs. For example one map using CCRR convention and another map using sequential numerical indexes. We do need ways of going from one convention to the other. Even better than that would be to have ways of going from *any* convention to *any* other. By "any" I mean, obviously, a feasible range of conventions where the information about location can be implicit.

The CTFS code (and `ctfs` + `fgeo` by inheritance) in particular uses two ways of identifying quadrats: quadrat code following CCRR convention, and quadrat index (numerical, consecutive). The quadrat index in particular is used often but rather internally; it is not part of the input of high-level functions.

- Note: to go from convention A to convention B, the easiest way is to simply use global coordinates. There is no need for a "translator" between quadrat naming conventions. The tutorials can have examples for this. The mandatory example is CCRR <-> sequential index exactly as done by CTFS code. I have to make sure that the new code returns the same codes and orders as the old CTFS code.

Despite the disproportionate complexity of the code to handle quadrats, coordinates, etc., there are very few direct dependents in CTFS and none in `fgeo`. The dependent functions in CTFS seem either trivial, easy to substitute, or directly removable. Clearly, most things down in the pipeline use global coordinates. There is no use whatsoever for local coordinates within CTFS. There is very limited use of different ways of identifying quadrats. Almost certainly, most of these functions were developed to compile the full data tables and not intended for anything else (despite a few later leaks into "ecological" code).

It is possible that universal adoption of tablets for fieldwork + standard good practices (for storing data and relate tables with each other) will make this code obsolete. In any case, and regardless of how well the code reads and work: **we should not encourage analytical workflows that depend on *assumed* locations of quadrats**. At a larger scale, it can be positive if we make that difficult and cumbersome for the user: no defaults, hard stops, and irritating warnings.

For that same reason, I feel it would be OK to ask for explicit quadrat location any time something is done at the quadrat level, with those locations NULL by default, and hard stops: "You must have explicit quadrat locations; if not, use function xxx". And then, have a solid function (or few) for one time use, to get locations implicit in quadrat IDs.

Aligned with this strict approach, any function that returns quadrat IDs must always include the quadrat locations as well. Therefore, functions that return complete quadrat-level tables

are preferred over functions that return more itemized outputs. This is aligned with the the general principle that things should make global sense.

6.3 Things should make global sense

The old code handles small pieces of information, maybe it's faster that way. But the different pieces of information work together: local coordinates don't make global sense without quadrat membership, and so on. The idea of **global sense** is probably important. Perhaps the output of all functions must be sufficient for it to make global sense.

Thing to consider:

- if input are coordinates, the output must contain coordinates too
- moving quadrat IDs around is not safe without objects that specify their location: either the user have them, or we must create them right away, or as often as possible.

Maybe accumulative outputs are safer than itemized outputs? For example, if input is global coordinates of trees, the output can be global coordinates of points + quadrat membership + local coordinates of points *and* also the quadrat-level table with (xmin, ymin, xmax, ymax) and attributes describing quadrat IDing convention. This was the approach taken by `fgeo` package, with those `add_*()` functions.

Do we want a "hard" definition of global? Probably not. It would make things more difficult, as we would need to carry global-level information as attributes or something like that. If we do not have a "hard global", splitting a plot into multiple subplots should be the same thing as splitting a smaller quadrat into multiple subquadrats. "Global" would mean "whatever you included", without a permanent reference to scales or objects that live outside the input. If that is the approach, **we should prioritize more general vocabulary like large/small, outer/inner, unit/subunit, etc. over global/local**.

6.4 Splitting

Splitting the plot into subplots, or quadrats into quadrats, is the same thing. The pieces can be identified by any IDing convention. A good default would be simply numbers, as one would use as a default for any grouping of observations.

Splitting the tree-level table based on global coordinates seems the most frequent or general case. But splitting polygons (plots, quadrats) based on (xmin, ymin, xmax, ymax) is also something that users may want to do (as in `getsmallerquads()` in CTFS).

The task of splitting seems something more "ecological" or useful for analyses than the other routines. Surprisingly, this was barely developed within CTFS. Splitting the tree data into

pieces can be useful for many things: summaries of local communities at different scales, beta-diversity at different scales, etc. There does not seem to be any reason to limit this flexibility at all, if we want users to be able to do very flexible ecological analyses.

We could think of different ways of splitting:

- into non-overlapping equal (probably square) pieces that fill the whole completely
- into non-overlapping equal (probably square) pieces with some smaller remainders (a warning seems enough)
- into overlapping equal (probably square) pieces? This is what the `selectrandomquad2()` function in CTFS was exploring, apparently.

This applies to subplots within the plot, subquadrats within quadrats, sub-sub-sub-sub-quadrats within sub-sub-sub-quadrats or whatever. Any resolution. Users may want to explore nested subdivisions of the plot to understand how things change with scale, and things like that. If we think in general terms, then users will be able to address more questions.

Naming pieces of a whole may be easy or difficult. The easiest way is to simply use a sequence of numbers. The IDing conventions should make it easy both to interpret existing IDs and to generate new IDs.

Splitting, in general, can be treated as arbitrary and resulting in temporary quadrats. Their locations don't need to be stored permanently, in general. However, **splitting + IDing convention is the way to recover quadrat codes** if the user lost them or if the user keeps quadrat-level information under different IDing conventions. Therefore, the splitting routines must be able to return quadrats neatly organized acoording to any given naming convention, and not just enforce simple numerical membership, for example. In fact, this is the only reason why splitting cannot be trivial in the new code, or simply omitted and substituted by teaching people how to group things into classes or bins in general.

The split of 20 x 20 m into 5 x 5 m quadrats is just one instance of a general routine to split. It does not require specific functions. We just have to make sure that the IDing 11, 12, 13, 13, 21, 22, etc. convention can be passed as an argument. This would cover the functionality of `lxly.to.p5()` in CTFS and `fgeo.tool::add_subquad()` and its dependencies. In this particular case, abstraction can save hundreds of lines of code and lots of intermediate functions.

6.5 Quadrat naming or IDing conventions

Need for generalization?

The conventions to identify quadrats seem central to the tasks that we need to solve. Conventions are declared within CTFS and fgeo code constantly, but there is not a fully fleshed system to handle them. The flexibility is limited to the variants of ForestGEO's most common

conventions.

If possible, we should be able to incorporate more flexibility. One of the goals of this package update is to reduce dependency on the "CTFS orthodox format" or other formal assumptions, so it can be more broadly usable (e.g. by participants in the Alliance for Tropical Forest Science).

If different conventions can be expressed in terms of {col, row} then we can re-use the calculations in CTFS. This must apply to any plot that has been divided into a grid, which is the most common practice.

Review of conventions

The quadrat IDing convention assumed by old CTFS code is relatively common but not universal, not even within ForestGEO. Many ForestGEO sites keep multiple versions of the data: one to store within STRI servers and orthodox, and another version they like more and use more often for more internal projects. And, of course, different projects can have different conventions to identify the quadrats or subplots.

This is a review of different ForestGEO datasets:

- BCI: QuadratName uses CCRR but immediately recognized as numeric by Excel. Starts at 0000. QuadratID seems to be numeric but it is imperfectly sequential (maybe internal to database). Some tables use 'CCRR with' first to avoid losing zeros through Excel. The convention for 5x5 m quadrats is CR within 20x20m quadrats (i.e. local meaning only), starting at 1, not 0.
  - The same convention seems to apply to other STRI smaller plots, with P09 or P16 plots, for example, numbering quadrats from 0000 to 0404, immediately losing initial zeros in Excel.
- Amacayacu: uses letters to identify hectares, then has quadrat codes of the form A01, A02, etc. Newer versions of the dataset use the form A1, A2, … without zeros. Each hectare (letter) has 25 quadrats.
  - Michigan Big Woods plot also identifies hectares by letter first, but the shape of the plot is irregular so there would not be a good way to infer their order. Hectare-level location in this plot must be explicitly kept.
- Yasuni: Uses CCRR or CC,RR for 20x20m quadrats, depending on the table, starting at 00. It uses CR for 5x5m quadrats, from 11 to 44.
- Fushan: In the {quadrat, habitat} object it uses simple numerical sequence from 1 to 625, referring to 20 x 20 m quadrats, which are also identified by global coordinates of their centroids. In other tables it uses CCRR starting at 0, from 0000 to 2424. In other tables, it uses CC,RR, with separator: 00,00 to 24,24. Some tables identify 10x10m subquadrats as "< v", "< ^", "> v", "> ^".

- Pasoh: Uses CCRR for 20x20 m quadrats, starting at 00. Sometimes with ' before to not to lose zeros. It also uses CR for 5x5 m subquadrats, from 11 to 44.
- Lambir: Uses CCRR from 0101 to 5225 for 20x20m quadrats.
- Khao Chong: Uses 10x10m quadrats using CCRR starting at 01, from 0101 to 6040 (the whole plot is 600 x 400 m). Initial zero can be lost by Excel.
- HKK: Uses CC-RR, starting at 01 for 20x20 m quadrats. In some tables, uses ' first to avoid losing zeros?. Within 20x20 m quadrats, there are four 10x10 quadrats labelled A, B, C, D.
- Rabi: Uses CCRR starting at 01 for 20x20m quadrats, from 0101 to 2525.
- Korup: Uses CCRR starting at 00 for 20x20m quadrats, from 0000 to 4924.
- Mpala: Uses CCRR from 0101 to 5025 in the tables I have. But the plot is longer than that: fgeo documentation mentions need for extra zero padding, so the forms CCCRR or CCCRRR are also feasible. It uses CR from 11 to 44 for 5x5m quadrats, but I have seen this only in field forms.
- Ngel Nyaki: Uses CCRR starting at 01 for 20x20m quadrats.
- Luquillo: Uses CCRR for 20x20 m quadrats, staring at 01, from 0101 to 1625 (the plot is 320 x 500 m). It also uses:
  - 10x10m subquadrast within 20x20m quadrats identified by cardinal directions: NW, NE, SW, SE.
  - 5x5m subquadrats within the 20x20m quadrats following CR, from 11 to 44.

Other networks may have different quadrat IDing conventions, but the variety of cases within ForestGEO datasets seems enough for the moment.

Axes of variation in quadrat IDing conventions

Based on the examples evaluated, and some other generalizations, we can identify the following axes of variation in quadrat IDing conventions:

- grid size = resolution of the quadrats (but possibly different along x and y)
- a simple sequence vs. a combination of row and col sub-ids.
- numbers vs. letters
- number of digits, i.e. whether do zero padding. Could be different for the different pieces that conform the quadrat ID.
- start at 0 or 1: initial zeros can disappear
- separators or initial characters to avoid losing zeros
- if a sequence, order in which things are numbered or named:
  - along columns first, then move to the next column
  - along rows first, then move to the next row
  - maybe changing directions: S to N for column 1, then N to S for column 2, etc. Idem with going along rows first.

  - arbitrary in practice: as in "< v", "< ^", "> v", "> ^", or "NW", "NE", "SW", "SE"
  - completly arbitrary: generalized unique IDs
- order in which the ID is built: e.g. CCRR vs. RRCC.

Although not present in the examples evaluated, we can anticipate cases of very systematic ways of naming quadrats, in GIS style:

- Corner-coordinate labels: subplot named by lower-left corner, e.g. x000_y020.
- Centroid-coordinate labels: subplot named by center point, e.g. x10_y10.

This might be covered with the other axes of variation as long as the numeric sequence can be declared in more detail, like in `seq(from = ..., to = ..., by = ...)` instead of simply `1:n`.

Nestedness and inheritance is something that requires particular attention. The subquadrats may have codes that only make sense within a given quadrat, or inherit the ID of the mother unit. For example, the standard codes for 5x5m quadrats don't inherit anything: the subquadrat-level codes only have local meaning. In contrast, in Amacayacu, the quadrats (sub-sub-units) inherit the code of the hectares (sub-units, relative to the whole plot). Similarly, we could anticipate things like 13A, 13-1, 13_1 for pieces of subplot "13", and so on. May refer to things like pieces of a map while digitizing, e.g. Map_0312_1, Map_0312_2, Map_0312_3, etc. When building IDs, this might require a `inherit = TRUE / FALSE` parameter or something equivalent? But that may limit the arbitrary application to sub-units within units. This is something to think about. Hopefully two-step sequential building / decoding is solid.

In any case, the best practice is to store quadrat IDs along their explicit locations (xmin, ymin, xmax, ymax). Extracting spatial meaning from quadrat IDs is risky and should remain a rare, infrequent task, to which the user has to pay proper attention. Therefore, we can be as detailed as we want in that step.

Multiple levels or ways of splitting the plot

There can be two (or more) ways of splitting the plot stored in the data. For example T1 / T2 levels in RAINFOR / Forestplots.net. Or the 20 x 20 m vs. 5 x 5 m quadrats systems. These could be nested levels or not. We should not assume nestedness in general. These different levels may have different naming conventions.

This should not be a problem, if naming conventions can be recorded well. This may be relevant for tutorials, more than the code itself. One or two or one hundred ways of splitting the plot should not make a big difference, as long as the IDs and features (locations) of those variables are clear.

Think about extensions

Current CTFS and fgeo code is impossible to adapt to new conventions. The new code needs to be readable, even if long and repetitive, so it is easy to expand with new conventions if needed. We can start by incorporating some conventions, and we can expand later.

Difficult cases

Cases outside the axes of variation above would be better encoded simply by storing {quadrat id, xmin, ymin, xmax, ymax}. Projects or networks with non-standard quadrat IDing conventions surely have that, even if just on a piece of paper in the wall. They can be treated as arbitrary IDs that require look up tables with explicit information about location, even if they have a system.

The non-rectangular cases may or may not be difficult to integrate. I won't do it for the moment:

- Strip labels: regeneration or understory units labelled as strip 1, strip 2, etc., not square quadrats.
- Transect-distance labels: subplots labelled by distance, e.g. 0–20 m, 20–40 m. Maybe referred by midpoint: 10, 30, etc.

6.6 Neighboring quadrats, etc.

The neighbor-related functions, according to the "functions cannot be examples" principle, will be probably substituted by:

- A function to indentify neighboring quadrats
  - possibly constrained (e.g. only neighbors of the same habitat, or same community type, or from groups from `hclust()`, etc.).
  - possibly through a "distance between quadrats" intermediate step, so we can generalize to k=4, k=8, etc., using k-NN routines.
- Tutorials or examples: abundances, presence/absence, basal area, …

As happens with the neighborhood analyses (of points): neighboring quadrats are just pieces of forest, subsets of trees, and the are many things that could be done with that, not just a couple.

6.7 Other dependents

Although not fully developed now, I should modify the following functions so they don't break:

- `fullplot.imageJ()`
- `allquadratslopes()`
- `selectrandomquad2()`
- `rasterize()`, part of the wavelet analysis
- `abundanceperquad()`, but consider direct removal
- `graph.mvnorm()`: depends on `full.xygrid()` = `base::expand.grid()` (apparently)

6.8 Not urgent

- quadrats are just polygons… Perhaps there are potential gains if we facilitate a polygon representation for quadrats. Probably enough by giving (xmin, ymin, xmax, ymax).

7 Summary

- There is a risky proliferation of functions to convert between pairs of conventions. The complexity of the code in `fgeo` in particular is out of proportion, involving dozens of functions for what are, essentially, very simple calculations. Things must be simplified radically, in terms of number of functions, number of dependencies, and length of code.
- There are two relevant scenarios: either the user keeps quadrat locations explicit (best practice) or implicit (bad practice). Getting and storing quadrat locations permanently is mandatory step for a well-managed project, so it should divide the funtions in two groups: those to get locations vs. those using locations.
- We cannot avoid the fundamental problem of guessing location from quadrat ID. For this to be general and usable outside ForestGEO's CCRR convention, we need powerful / detailed ways of handling naming or IDing conventions. This would be new complexity. Ideally, it should be easy to adapt and expand.
- Splitting can be general and easy. But it is also the key to recovering quadrat IDs: it must handle naming conventions and order of quadrats with care.
- Things should try to make global sense. Any time a function splits a set of coordinates into pieces, it must return piecel-level ID, coordinates within pieces, piece locations, and, probably, the corresponding naming convention as an attribute.

- Neighboring quadrats: not key, not in `fgeo`. Probably can be substituted by a tutorial on k-NN applied to quadrats, which is more general, with the tasks of the CTFS functions as examples: abundances, proportion of presences, …
- Other dependents: make small modifications to ensure things don't break, and review in more detail when their time arrives.

8 Clean up

```
# Remove the temporary folder we created to
# access the project at a specific point in history:
knitr::opts_knit$set(root.dir = path)
if(dir.exists(historical_path))
  remove_dir_to_point_in_history(path, historical_path)

dir.exists(historical_path) # should be FALSE now
```

```
[1] FALSE
```